C.E.S.A.R school

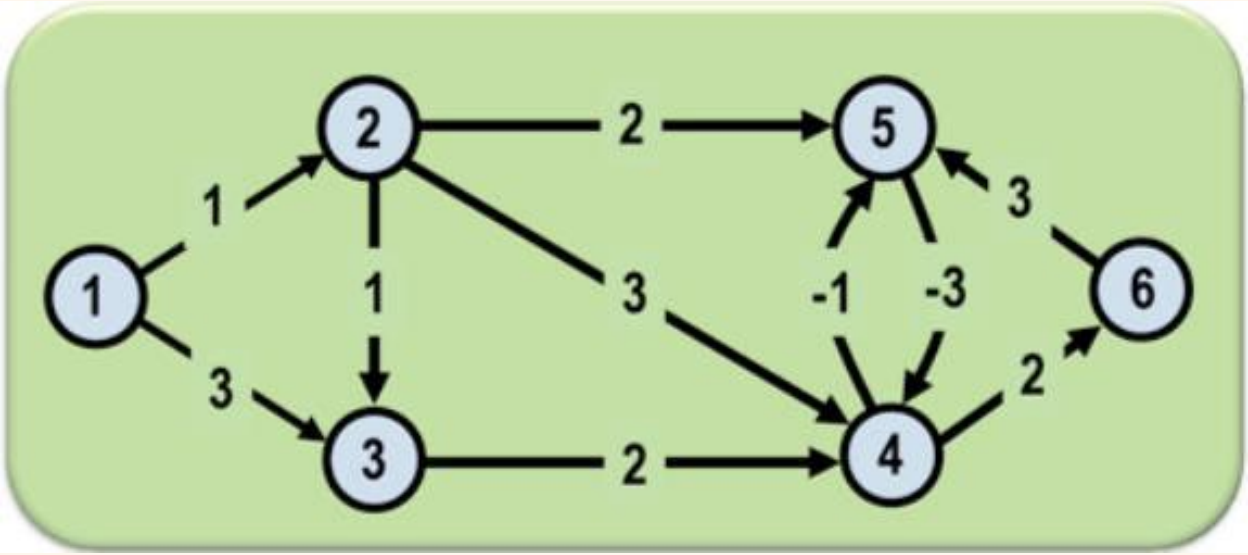
TEORIA DOS GRAFOS

Profª Laura Pacifico

2026 | AGOSTO

Atividade

Utilizando o algoritmo de Bellman-Ford, calcule a menor distância entre os vértices 1 e 6 (determine o caminho). Vamos corrigir juntos!



O Problema do Caminho Mínimo

Em um grafo ponderado, cada aresta possui um **peso** que representa custo, distância ou tempo. O objetivo é encontrar o caminho entre vértices cujo **custo total seja mínimo**.

Redes de Computadores

Roteamento eficiente de pacotes

Sistemas de Navegação

GPS e rotas otimizadas

Logística e Transporte

Minimização de custos operacionais

Análise de Redes

Influência e alcance em grafos

Tipos de Problemas de Caminho Mínimo

Dependendo da estrutura do problema, há três variações fundamentais:

1

Single Source Shortest Path

Menor caminho de um vértice origem para **todos os outros**. Exemplos: **Dijkstra** e **Bellman-Ford**.

2

Single Pair Shortest Path

Menor caminho entre **dois vértices específicos**. Pode ser derivado do caso anterior com parada antecipada.

3

All Pairs Shortest Path

Menores caminhos entre **todos os pares** de vértices. Exemplo: **Floyd-Warshall**.

Conceito Fundamental: Relaxamento

Relaxar uma aresta significa verificar se existe um caminho **mais curto** passando por ela. É a operação central de todos os algoritmos de caminhos mínimos.

> Se **d(v) > d(u) + w(u,v)**, então atualizamos **d(v) = d(u) + w(u,v)**.

Intuitivamente: se chegar a **v** via **u** for mais barato do que o custo estimado atual, a estimativa deve ser melhorada.

Intuição do Relaxamento

Um exemplo concreto para fixar o conceito:

Distâncias conhecidas

dist(A, B) = **3**, dist(B, C) = **2**

Conclusão imediata

dist(A, C) ≤ **3 + 2 = 5**

Relaxamento

Se dist(A, C) atual **> 5**, podemos melhorar a estimativa via B.

Esse processo se propaga por todo o grafo, melhorando progressivamente as estimativas de distância a cada iteração.

Propagação das Distâncias

O relaxamento permite que **informações sobre caminhos melhores se propaguem progressivamente** pelo grafo. Estimativas começam superestimadas e convergem ao valor ótimo.

Inicialização

Distância da origem = 0; todos os outros = ∞

Propagação

Cada relaxamento melhora a estimativa de um vizinho

Convergência

Após iterações suficientes, todas as distâncias são ótimas

Dijkstra: Um Algoritmo Guloso

Um algoritmo é **guloso (greedy)** quando escolhe, a cada passo, a opção que parece melhor no momento, sem jamais reconsiderar decisões já tomadas.

> A escolha gulosa: sempre o **vértice não visitado com menor distância estimada**.

Essa decisão nunca é reconsiderada depois por isso Dijkstra é guloso, e por isso falha com pesos negativos.

Pseudocódigo: Dijkstra

Busca gulosa pelo vértice não visitado mais próximo, a partir da origem.

```
DIJKSTRA(G, w, s)
  para cada vértice v de G:
      d(v) ← ∞
      visitado(v) ← falso
  d(s) ← 0

  enquanto houver vértice não visitado:
      u ← vértice não visitado com menor d(u)
      visitado(u) ← verdadeiro
      para cada aresta (u, v) de peso w(u,v):
          se d(v) > d(u) + w(u,v):
              d(v) ← d(u) + w(u,v)
              anterior(v) ← u

  retorna d, anterior
```

Limitações do Algoritmo de Dijkstra

⚠️ Dijkstra **não funciona corretamente** na presença de pesos negativos.

O algoritmo assume que, ao finalizar o processamento de um vértice, sua distância é definitiva e nunca será reduzida.

Com pesos negativos, essa suposição **deixa de ser válida**: um caminho mais longo em número de arestas pode ter custo total menor, e Dijkstra nunca o reconsideraria.

Exemplo: Onde Dijkstra Falha

O Grafo

A → B = **2**

A → C = **5**

C → B = **–10**

O Problema

Dijkstra escolhe **B primeiro** com custo 2, marcando-o como definitivo.

Porém o caminho real mais curto é:

A → C → B = 5 + (–10) = –5

Dijkstra retorna **2**. O valor correto é **–5**. Resultado incorreto.

Ciclos Negativos

Um **ciclo negativo** ocorre quando a soma dos pesos das arestas de um ciclo é **menor que zero**.

Exemplo de Ciclo

A → B = 2

B → C = –5

C → A = 1

Peso Total

2 + (–5) + 1 = **–2**

Cada volta pelo ciclo reduz o custo em 2 unidades.

Por que Ciclos Negativos São um Problema?

Percorrendo o ciclo repetidamente, o custo cai indefinidamente: −2, −4, −6, −8...

Não existe caminho mínimo definido em grafos com ciclos negativos alcançáveis.

Pseudocódigo: Bellman-Ford

Relaxamento de todas as arestas, repetido |V| − 1 vezes, com checagem de ciclo negativo.

```
BELLMAN-FORD(G, w, s)
  para cada vértice v de G:
      d(v) ← ∞
      anterior(v) ← nulo
  d(s) ← 0

  repita (|V| - 1) vezes:
      para cada aresta (u, v) de peso w(u,v):
          se d(v) > d(u) + w(u,v):
              d(v) ← d(u) + w(u,v)
              anterior(v) ← u

  // checagem de ciclo negativo, uma rodada extra:
  para cada aresta (u, v) de peso w(u,v):
      se d(v) > d(u) + w(u,v): retorna "ciclo negativo"
```

Como Bellman-Ford Detecta Ciclos Negativos

O Bellman-Ford realiza relaxamentos repetidos e usa uma propriedade matemática para detectar ciclos negativos.



A chave está na **iteração adicional**: em grafos sem ciclos negativos, as distâncias já convergiram após |V|−1 rodadas. Qualquer melhoria adicional revela um ciclo.

Intuição da Detecção

Por que |V|–1 iterações bastam?

O caminho simples mais longo em um grafo com **V** vértices possui no máximo **|V|–1** arestas. Após essas iterações, **todas as distâncias convergiram**.

A prova por contradição

Se após a **|V|-ésima iteração** uma distância ainda diminui, o caminho teria **|V| arestas**, implicando um **ciclo**. Como esse ciclo reduz custo, é necessariamente negativo.

Comparação entre Algoritmos

Algoritmo	Pesos Negativos	Complexidade	Caso de Uso
Dijkstra	Não	$O(E \log V)$	Grafos esparsos, pesos positivos
Bellman-Ford	Sim	$O(V \cdot E)$	Detecção de ciclos negativos
Floyd-Warshall	Sim	$O(V^3)$	Todos os pares de vértices

A escolha do algoritmo correto depende da **estrutura do grafo** e do **tipo de consulta** desejado.

Outros Algoritmos de Caminhos Mínimos

Além de Dijkstra e Bellman-Ford, outras abordagens resolvem variações do problema:

1

Floyd-Warshall

Programação dinâmica que calcula a distância entre **todos os pares** de vértices, em $O(V^3)$.

2

A* (A-star)

Usa uma **heurística** para guiar a busca até o destino, acelerando Dijkstra em consultas ponto a ponto.

3

Algoritmo de Johnson

Combina Bellman-Ford e Dijkstra para resolver **todos os pares** em grafos esparsos.

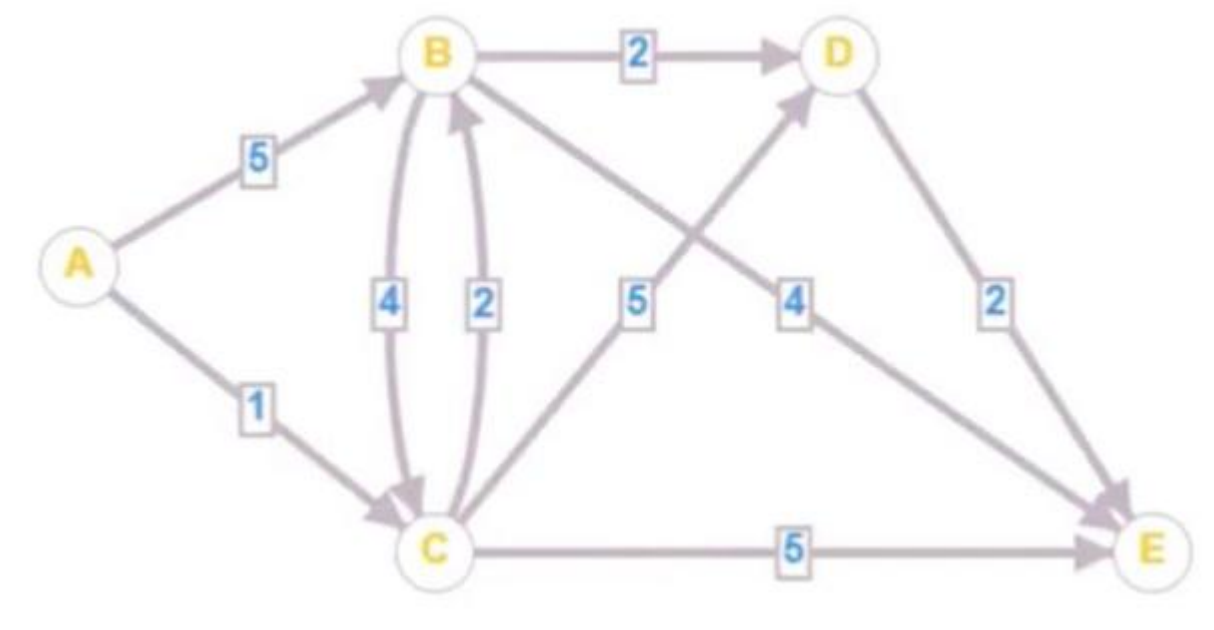
Diferença de Desempenho na Prática

Escala comparativa (operações estimadas)

- **Dijkstra** domina em grafos grandes com pesos positivos
- **Bellman-Ford** é necessário quando há pesos negativos
- **Floyd-Warshall** só é prático para grafos pequenos ou densos

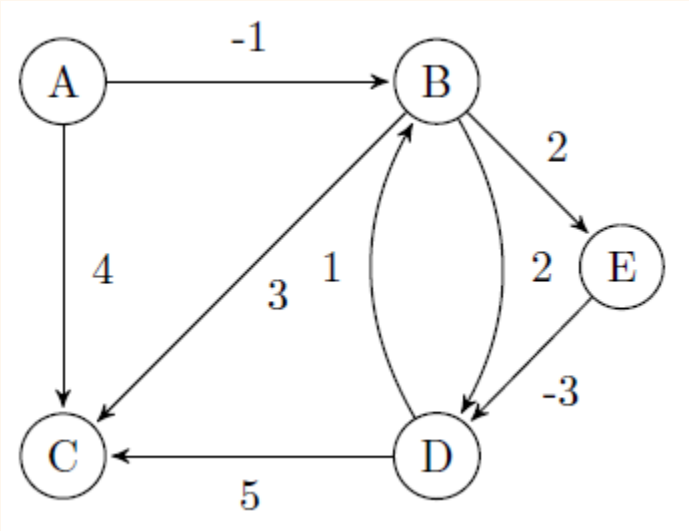
Exercício

Use o algoritmo de Dijsktra para encontrar o menor caminho a partir da origem A até o vértice E.



Exercício

Use o algoritmo de Belman-Ford para encontrar o menor caminho a partir da origem A até os demais vértices do grafo.



Dúvidas?



Laura Alves Pacifico
laps@cesar.school
Slack: Laura Pacifico